

porque foi usado a função componentDidMount() ?

A função `componentDidMount()` é um método do ciclo de vida de componentes React que é chamado automaticamente após um componente ser renderizado pela primeira vez no DOM (árvore de elementos da página web).

Por que usar `componentDidMount()` para buscar dados?

1. **Lugar ideal para operações assíncronas:** A busca de dados geralmente envolve operações assíncronas, como chamadas de API ou solicitações de rede. O `componentDidMount()` é o local ideal para realizar essas operações, pois garante que o componente já foi montado no DOM antes de iniciar a busca.
2. **Manipulação do DOM:** Em alguns casos, você pode precisar interagir diretamente com elementos do DOM após o componente ser renderizado. O `componentDidMount()` garante que os elementos estejam disponíveis para manipulação.
3. **Integração com bibliotecas externas:** Se você estiver usando bibliotecas externas que dependem do DOM, como plugins jQuery ou outras ferramentas de manipulação de DOM, o `componentDidMount()` é o lugar certo para inicializá-las.

Exemplo:

JavaScript

```
import React from 'react';

class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      data: null,
      loading: true,
    };
  }

  componentDidMount() {
    // Simula uma chamada de API
    fetch('/api/data')
      .then(response => response.json())
      .then(data => {
        this.setState({ data: data, loading: false });
      });
  }
}
```

```

render() {
  if (this.state.loading) {
    return <div>Carregando...</div>;
  }

  return (
    <div>
      {/* Exibe os dados */}
    </div>
  );
}
}

```

Observações:

- O `componentDidMount()` é chamado apenas uma vez, após a primeira renderização do componente.
- Se você precisar buscar dados novamente quando as props do componente mudarem, use o método `componentDidUpdate()`.
- Em componentes funcionais com React Hooks, o equivalente ao `componentDidMount()` é o hook `useEffect()` com um array de dependências vazio:

JavaScript

```

import React, { useState, useEffect } from 'react';

function MyComponent() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Simula uma chamada de API
    fetch('/api/data')
      .then(response => response.json())
      .then(data => {
        setData(data);
        setLoading(false);
      });
  }, []); // O array vazio garante que o efeito seja executado apenas uma vez

  if (loading) {
    return <div>Carregando...</div>;
  }
}

```

```
return (  
  <div>  
    { /* Exibe os dados */ }  
  </div>  
) ;  
}
```

Espero que esta explicação detalhada tenha sido útil! Se tiver mais dúvidas, fique à vontade para perguntar.

Fontes

1. <https://blog.openreplay.com/abort-controllers-in-javascript/>